Programação Orientada a Objetos

ALOCAÇÃO DINÂMICA DE MEMÓRIA

em C++

Introdução

- A alocação dinâmica de memória
  - É feita durante a execução do programa
  - Utiliza new e delete

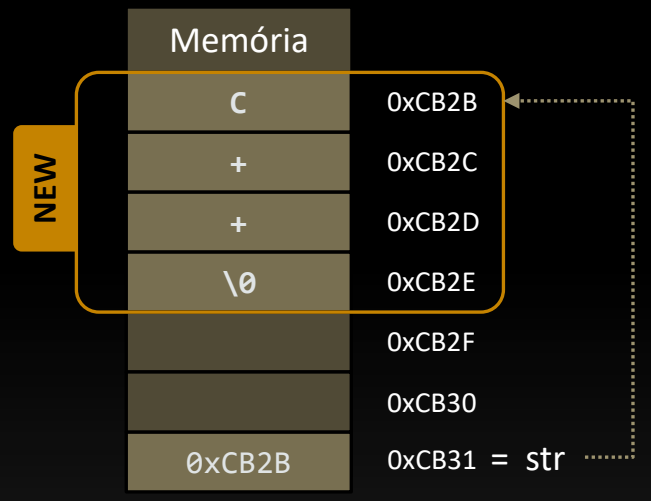
```
int main()
{
    char nome[40];                  // alocação automática
                                    // tempo de compilação
    int tam;
    cin >> tam;
    char * str = new char[tam];     // alocação dinâmica
                                    // tempo de execução
    delete [] str;
}
```

Introdução

- A alocação dinâmica de memória
  - Possui vantagens:
    - Não desperdiça memória
    - Tem toda a RAM disponível *

```
char * str = new char[tam];
```

Quantidade pode ser calculada durante a execução

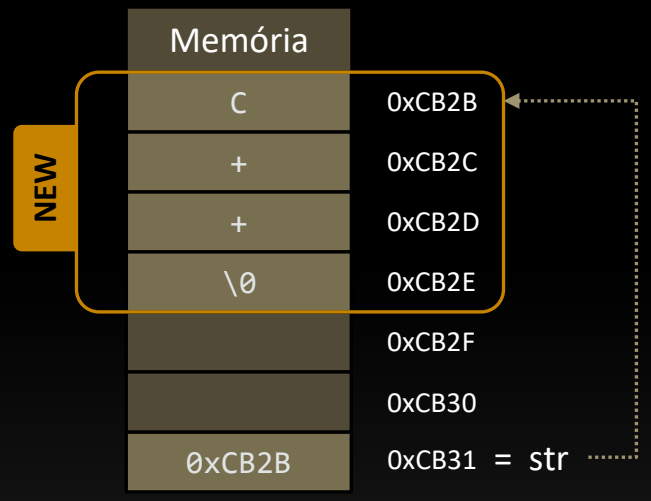


Introdução

- A alocação dinâmica de memória
  - Possui desvantagens:
    - Alocar antes de usar
    - Liberar a memória ao final do uso
    - Não utilizar após a liberação

```
delete [] str;
```

Libera memória, não destrói o ponteiro

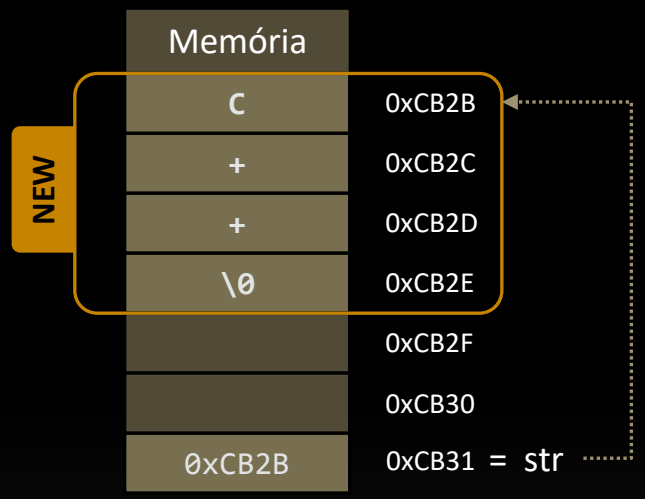


Gerenciamento de Memória

- As classes eliminam as desvantagens
  - Elas fazem o gerenciamento da memória
    - Construtor: alocação de memória
    - Destrutor: liberação

```
// construtor
String::String(const char txt[])
{
    tam = strlen(txt);
    str = new char[tam + 1];
    strpcy(str, txt);
}
```

```
// destrutor
String::~String()
{
    delete [] str;
}
```



Gerenciamento de Memória

- O gerenciamento passa a ser automático
  - Criação do objeto
  - Destruição do objeto

```
int main()
{
    String nome { "C++" };
}
```

```
// construtor
String::String(const char txt[])
{
    tam = strlen(txt);
    str = new char[tam + 1];
    strpcy(str, txt);
}
```

```
// destrutor
String::~String()
{
    delete [] str;
}
```

Gerenciamento de Memória

- Acontece dentro da classe
  - Atributos privados
  - Detalhes internos

```
class String
{
private:
    char *  str ;
    size_t  tam ;

    ...
};
```

```
// construtor
String::String(const char txt[])
{
    tam  = strlen(txt);
    str  = new char[tam + 1];
    strpcy(str, txt);
}
```

```
int main()
{
    String nome { "C++" };
}
```

```
// destrutor
String::~String()
{
    delete [] str;
}
```

Membros Estáticos

- Registrar o número de instâncias

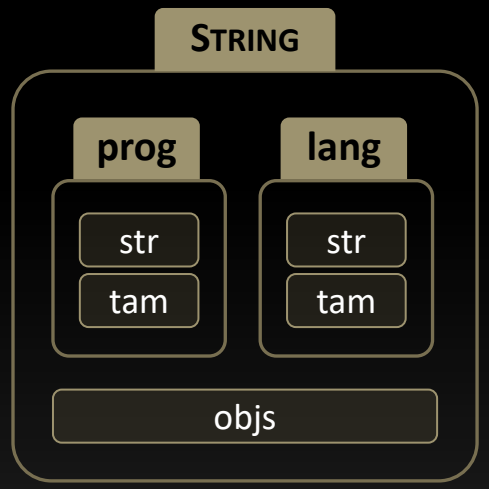
```
class String
{
private:
   char * str;        // ponteiro para string
   size_t tam;        // tamanho da string

   static int objs;   // número de instâncias

   ...
};


int main()
{
    String prog { "Programação" };
    String lang { "C++" };
}
```

```
// inicialização de membro estático
int String::objs = 0;

// construtor
String::String(const char txt[])
{
    tam = strlen(txt);
    str = new char[tam + 1];
    strpcy(str, txt);

    objs++;
}
```

Membros Estáticos

- Isso acontece porque:
  - Membro estático: pertence a classe
  - Atributos: pertence ao objeto

```
int main()
{
    String prog { "Programação" };
    String lang { "C++" };

    // existem 2 objetos
    // mas apenas um membro estático
    // objs = 2
}
```




```
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

   ...
};
```

A Classe String

- Construindo um exemplo:

```
class String
{
private:
   char * str;                    // ponteiro para string
   size_t tam;                    // tamanho da string
   static int objs;               // número de instâncias

public:
   String(const char txt[]);      // construtor
   String();                      // construtor padrão
   ~String();                     // destrutor

   // função amiga para exibir String com cout
   friend ostream& operator<<(ostream& os, const String& s);
};
```

» String.h

A Classe String

```cpp
#include "String.h"
#include <cstring>

int String::objs = 0;

String::String(const char txt[])
{
    tam = strlen(txt);
    str = new char[tam + 1];
    strpcy(str, txt);
    objs++;
}

String::String()
{
    tam = 0;
    str = new char[1] { "" };
    objs++;
}
```

```cpp
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

public:
   String(const char txt[]);
   String();
   ~String();

   friend ostream& operator<<(
      ostream& os, const String& s);
};
```

» String.cpp

A Classe String

```
...

String::~String()
{
    delete [] str;
    objs--;
}

ostream& operator<<(
   ostream& os,
   const String& s)
{
    os << s.str;
    return os;
}
```

```
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

public:
   String(const char txt[]);
   String();
   ~String();

   friend ostream& operator<<(
      ostream& os, const String& s);
};
```

» String.cpp

A Classe String

```
#include <iostream>
#include "String.h"
using std::cout;
using std::endl;

int main()
{
    cout << "Entrando no bloco\n\n";
    {
        String games { "Asha Sharman nomeada CEO do Xbox" };
        String techn { "Netflix desiste de comprar a Warner" };
        String sport { "Jutta Leerdam quebra recorde no Speed Skating" };

        cout << games << endl;
        cout << techn << endl;
        cout << sport << endl;
    }
    cout << "\nSaindo do bloco\n";
}
```

» Noticias.cpp

A Classe String

- Saída do programa:

```
Entrando no bloco

Asha Sharman nomeada CEO do Xbox
Netflix desiste de comprar a Warner
Jutta Leerdam quebra recorde no Speed Skating

Saindo do bloco
```

- Foram criadas 3 instâncias da classe String

| Objeto (1) criado: "Asha Sharman nomeada CEO do Xbox"
| Objeto (2) criado: "Netflix desiste de comprar a Warner"
| Objeto (3) criado: "Jutta Leerdam quebra recorde no Speed Skating"

Resumo

- As classes simplificam a alocação dinâmica
  - Construtor: alocação da memória
  - Destrutor: liberação da memória
- O gerenciamento de memória
  - Fica oculto e encapsulado
  - Atributos privados
- Membros estáticos
  - Pertencem a classe e não aos objetos
  - Mantêm o seu valor ao longo do programa